

THOTH CLOUD

Plateforme Cloud Multi-Services

IaaS / PaaS / SaaS

Rapport d'Implémentation Complet

TP4 : Évolution vers un Mini Cloud PaaS

INF4107

Virtualisation et Cloud Computing

Master 1 - Systèmes & Réseaux

Année Académique 2025-2026

Étudiant : EDU GUIEDI HERMANN ARNOLD
Matricule : 22T2876
Enseignants : Dr. Valery MONTHE & M. Gildas NGOUANFO
Date : 5 février 2026

Dépôt GitHub du Projet

https://github.com/InputStream/tp_kvm

Table des matières

1	Introduction	4
1.1	Contexte du Projet	4
1.2	Objectifs du TP4	4
1.3	Technologies Utilisées	5
1.4	Environnement de Travail	5
2	Architecture Globale	5
2.1	Vue d'Ensemble du Système	5
2.2	Architecture Backend	6
2.3	Architecture Frontend	6
3	Partie 1 : Amélioration du Service IaaS	7
3.1	Multi-Tenant avec Isolation Utilisateur	7
3.1.1	Système de Nommage des VMs	7
3.1.2	Filtrage des Ressources par Utilisateur	7
3.2	Découplage via Conteneurisation	8
3.2.1	Dockerfile	8
3.2.2	Communication avec Hôtes KVM Distants	8
3.3	Isolation Réseau	8
3.3.1	Réseau par Utilisateur	8
3.3.2	Réseau par Cluster Docker Swarm	9
3.4	Système de Flavors	9
3.4.1	Définition des Flavors	9
3.4.2	Vérification de Capacité	10
3.5	Gestion Multi-Hôtes	10
3.5.1	Configuration des Hôtes	10
3.5.2	Stratégie Least Used (LRU)	11
3.5.3	Autres Stratégies Proposées	11
4	Partie 2 : Déploiement Automatique de Clusters Docker Swarm	12
4.1	Architecture du Cluster Swarm	12
4.1.1	Configuration Type	12
4.2	Déploiement Automatisé des VMs	12
4.2.1	Script de Déploiement	12
4.3	Configuration Automatique via Cloud-Init	13
4.3.1	User-Data pour Installation Docker	13
4.3.2	Configuration des Workers	13
4.4	Ports Nécessaires	14
4.5	Affichage des Informations du Cluster	14
5	Partie 3 : Service PaaS - Catalogue d'Applications	14
5.1	Catalogue d'Applications	14
5.1.1	Liste des Applications	14
5.2	Mutualisation des Bases de Données	15
5.2.1	Déploiement des SGBD Mutualisés	15
5.2.2	Création de Base par Application	15

5.3	Déploiement en 1 Clic	16
5.3.1	Processus de Déploiement	16
5.3.2	Exemple : Déploiement WordPress	16
5.4	Système de Facturation	17
5.4.1	Collecte des Métriques	17
5.4.2	Calcul du Coût	17
6	Partie 4 : Service SaaS - Plateforme Collaborative	18
6.1	Architecture OwnCloud + OnlyOffice	18
6.1.1	Fonctionnalités Proposées	18
6.2	Déploiement sur Docker Swarm	18
6.2.1	Docker Compose	18
6.3	Intégration OwnCloud - OnlyOffice	19
6.3.1	Configuration de l'Application OnlyOffice dans OwnCloud	19
6.4	Facturation au Stockage	20
6.4.1	Grille Tarifaire	20
6.4.2	Calcul de la Consommation	20
7	Implémentation Frontend	20
7.1	Structure de l'Interface	20
7.2	Authentification et Session	21
7.2.1	Système JWT	21
7.2.2	Protection des Routes	21
7.3	Gestion des VMs	21
7.3.1	Affichage des VMs	21
7.4	Déploiement de VMs avec Flavors	22
7.5	Catalogue PaaS	23
7.5.1	Affichage du Catalogue	23
7.6	Thème Clair/Sombre	23
8	Guide de Déploiement	24
8.1	Prérequis	24
8.2	Installation	24
8.2.1	Étape 1 : Cloner le Dépôt	24
8.2.2	Étape 2 : Configuration	24
8.2.3	Étape 3 : Compilation du Backend	24
8.2.4	Étape 4 : Déploiement avec Docker	25
8.3	Accès à l'Interface	25
8.4	Configuration Post-Installation	25
9	Résultats et Démonstrations	25
9.1	Partie 1 : IaaS Multi-Tenant	25
9.2	Partie 2 : Clusters Docker Swarm	26
9.3	Partie 3 : PaaS et Catalogue	26
9.4	Partie 4 : SaaS Collaboratif	26

10 Limitations et Améliorations	26
10.1 Limitations Connues	26
10.2 Améliorations Proposées	27
10.2.1 Court Terme (1-3 mois)	27
10.2.2 Moyen Terme (3-6 mois)	27
10.2.3 Long Terme (6-12 mois)	28
11 Conclusion	28
11.1 Bilan de Réalisation	28
11.2 Compétences Développées	28
11.3 Perspectives d'Évolution	29
11.4 Remerciements	29
A Structure du Code Source	29
B Configuration Système	30
B.1 Fichier de Configuration Principal	30
B.2 Configuration des Hôtes	31
C Commandes Utiles	31
C.1 Gestion des VMs	31
C.2 Gestion Docker Swarm	32
D Captures d'Écran	33

1 Introduction

1.1 Contexte du Projet

THOTH CLOUD est une plateforme de gestion cloud complète développée dans le cadre du TP4 d'INF4107. Le projet constitue une évolution majeure du Mini Cloud IaaS développé lors du TP3, en ajoutant des couches PaaS et SaaS pour offrir une solution cloud complète et multi-services.

Évolution du Projet **TP3 (Base IaaS)** : Provisionnement de VMs via interface web avec monitoring basique

TP4 (Cloud Complet) : IaaS multi-tenant + PaaS avec Docker Swarm + SaaS collaboratif

Le projet vise à créer un mini cloud offrant trois niveaux de services :

- **IaaS** (Infrastructure as a Service) : Provisionnement et gestion de machines virtuelles avec isolation multi-utilisateurs, gestion multi-hôtes, flavors et facturation
- **PaaS** (Platform as a Service) : Déploiement d'applications conteneurisées via Docker Swarm avec catalogue d'applications en "1 clic"
- **SaaS** (Software as a Service) : Plateforme collaborative de type Google Workspace basée sur OwnCloud + OnlyOffice

1.2 Objectifs du TP4

Les objectifs principaux de ce travail pratique sont :

1. **Améliorer le service IaaS** (Partie 1) :
 - Implémenter l'isolation multi-utilisateurs
 - Découpler l'application de l'hôte KVM via conteneurisation
 - Mettre en place l'isolation réseau par utilisateur
 - Définir un système de flavors avec pricing
 - Gérer plusieurs hôtes KVM avec stratégie de répartition
2. **Déployer automatiquement des clusters Docker Swarm** (Partie 2) :
 - Permettre aux utilisateurs de créer leurs clusters à la demande
 - Automatiser l'installation et la configuration via cloud-init/Ansible
 - Gérer les rôles (managers et workers)
3. **Fournir un service PaaS** (Partie 3) :
 - Créer un catalogue d'applications déployables en 1 clic
 - Mutualiser les bases de données PostgreSQL et MariaDB
 - Implémenter un système de facturation basé sur la consommation
4. **Proposer un service SaaS** (Partie 4) :
 - Déployer une plateforme collaborative (OwnCloud + OnlyOffice)
 - Fournir une alternative à Google Workspace
 - Implémenter une facturation basée sur le stockage

1.3 Technologies Utilisées

Catégorie	Technologies
Backend	C++17, cpp-httplib, nlohmann/json
Frontend	HTML5, CSS3, JavaScript ES6+ (SPA)
Virtualisation	KVM/QEMU, libvirt
Conteneurisation	Docker, Docker Swarm
Orchestration	cloud-init, Ansible
Bases de données	PostgreSQL, MariaDB
Réseau	libvirt networks, iptables
Monitoring	REST API, WebSocket (temps réel)

TABLE 1 – Stack technologique du projet

1.4 Environnement de Travail

- **Système hôte** : Ubuntu 22.04 LTS
- **Hyperviseur** : KVM/QEMU avec support Intel VT-x / AMD-V
- **Ressources** : Plusieurs hôtes KVM pour démontrer la gestion multi-hôtes
- **Réseau** : Configuration de réseaux virtuels isolés par utilisateur
- **Stockage** : /var/lib/libvirt/images pour les disques virtuels et images

2 Architecture Globale

2.1 Vue d'Ensemble du Système

THOTH CLOUD adopte une architecture en couches modulaire permettant une séparation claire des responsabilités :

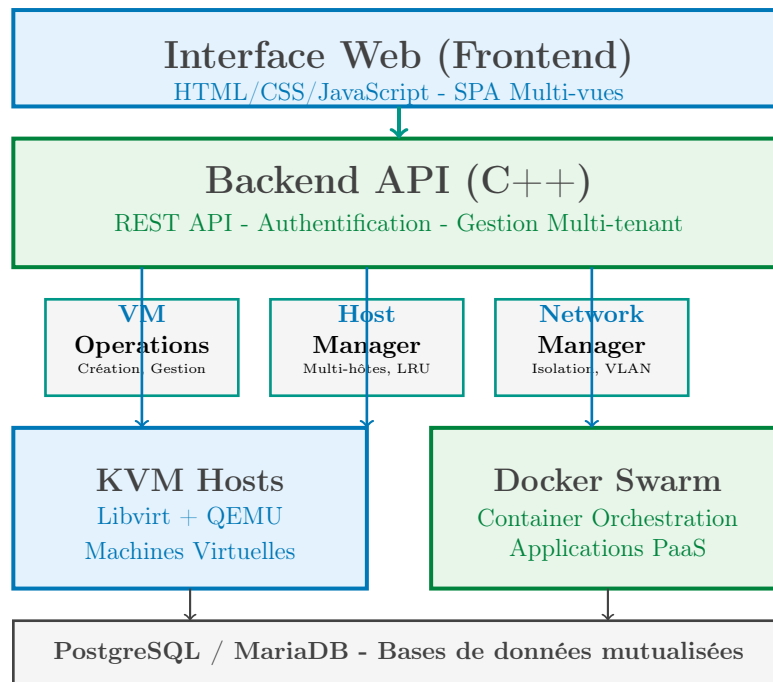


FIGURE 1 – Architecture en couches de THOTH CLOUD

2.2 Architecture Backend

Le backend est développé en C++17 avec une architecture modulaire comprenant :

- **API REST** (cpp-http-lib) : Serveur HTTP exposant les endpoints pour toutes les opérations
- **Gestionnaire d'utilisateurs** : Authentification JWT, isolation des ressources par utilisateur
- **Gestionnaire de VMs** : Création, démarrage, arrêt, suppression avec nommage unique par utilisateur
- **Gestionnaire d'hôtes** : Gestion multi-hôtes KVM avec stratégies de répartition (LRU, Round Robin, Random)
- **Gestionnaire de réseaux** : Création de réseaux isolés par utilisateur et par cluster
- **Gestionnaire de flavors** : Définition d'offres de ressources avec pricing
- **Gestionnaire PaaS** : Déploiement d'applications sur Docker Swarm
- **Gestionnaire de facturation** : Calcul des coûts basé sur la consommation

2.3 Architecture Frontend

L'interface web est une Single Page Application (SPA) moderne comprenant :

- **Système d'authentification** : Pages login/register avec gestion de session JWT
- **Navigation par vues** : Système de routing côté client sans rechargement de page
- **Tableau de bord** : Vue d'ensemble des ressources utilisateur (VMs, apps, clusters)
- **Gestion des VMs** : Création, monitoring, actions (start/stop/clone/delete)
- **Catalogue PaaS** : Interface de déploiement d'applications en 1 clic

- **Gestion Swarm** : Visualisation et gestion des clusters Docker
- **Interface SaaS** : Accès à OwnCloud + OnlyOffice
- **Facturation** : Visualisation de la consommation et des coûts

3 Partie 1 : Amélioration du Service IaaS

3.1 Multi-Tenant avec Isolation Utilisateur

Objectif Atteint Chaque utilisateur dispose de ressources complètement isolées et ne peut accéder qu'à ses propres VMs, réseaux et clusters Docker Swarm.

3.1.1 Système de Nommage des VMs

Pour garantir l'unicité et l'isolation, nous avons implémenté un système de nommage structuré :

```
1 std::string createVMName(const std::string& username,
2                          const std::string& userVMName) {
3     std::string cleanUser = sanitize(username);
4     std::string cleanVMName = sanitize(userVMName);
5     std::string timestamp = getTimestamp();
6
7     // Format: username__vmname__timestamp
8     return cleanUser + "__" + cleanVMName + "__" + timestamp;
9 }
```

Listing 1 – Génération de noms de VMs (utils.hpp)

Avantages du système :

- Unicité garantie grâce au timestamp
- Identification claire de l'utilisateur propriétaire
- Traçabilité de la date de création
- Prévention des conflits de noms entre utilisateurs

Exemple : Un utilisateur `alice` créant une VM `webserver` obtiendra :
`alice__webserver__20260205143022`

3.1.2 Filtrage des Ressources par Utilisateur

Toutes les opérations de listage filtrent les ressources selon l'utilisateur connecté :

```
1 std::vector<VM> getUserVMs(const std::string& username) {
2     std::vector<VM> userVMs;
3     std::vector<VM> allVMs = listAllVMs();
4
5     std::string prefix = username + "__";
6     for (const auto& vm : allVMs) {
7         if (vm.name.find(prefix) == 0) {
8             userVMs.push_back(vm);
9         }
10    }
11    return userVMs;
}
```


12 }

Listing 2 – Filtrage des VMs par utilisateur

3.2 Découplage via Conteneurisation

Architecture Découplée L'application backend ne s'exécute plus directement sur l'hôte KVM mais dans un conteneur Docker, communiquant avec les hôtes KVM distants via libvirt.

3.2.1 Dockerfile

```
1 FROM ubuntu:22.04
2
3 RUN apt-get update && apt-get install -y \
4     build-essential \
5     cmake \
6     libvirt-dev \
7     libssh2-1-dev \
8     nlohmann-json3-dev \
9     && rm -rf /var/lib/apt/lists/*
10
11 WORKDIR /app
12 COPY . .
13 RUN mkdir build && cd build && cmake .. && make
14
15 EXPOSE 3000
16 CMD ["/build/thoth-cloud"]
```

Listing 3 – Dockerfile du backend

3.2.2 Communication avec Hôtes KVM Distants

Le backend peut gérer plusieurs hôtes KVM via différents URIs libvirt :

```
1 // H te local
2 virConnectPtr conn1 = virConnectOpen("qemu:///system");
3
4 // H te distant via SSH
5 virConnectPtr conn2 = virConnectOpen(
6     "qemu+ssh://user@192.168.1.100/system"
7 );
```

Listing 4 – Connexion multi-hôtes

3.3 Isolation Réseau

3.3.1 Réseau par Utilisateur

Chaque utilisateur obtient son propre réseau libvirt isolé :

```

1 void createUserNetwork(const std::string& username) {
2     std::string networkName = "net_" + username;
3     std::string subnet = allocateSubnet(username); // Ex:
4     192.168.10.0/24
5
6     std::string xmlConfig = R"(
7         <network>
8             <name>>" + networkName + R"(</name>
9             <forward mode='nat' />
10            <ip address='>" + subnet + R"(' netmask='255.255.255.0'>
11                <dhcp>
12                    <range start='>" + subnet + R"(.10'
13                        end='>" + subnet + R"(.254' />
14                </dhcp>
15            </ip>
16        </network>
17    )";
18
19    virNetworkPtr network = virNetworkDefineXML(conn, xmlConfig.c_str())
20    ;
21    virNetworkCreate(network);
22    virNetworkSetAutostart(network, 1);
23 }

```

Listing 5 – Création de réseau utilisateur

3.3.2 Réseau par Cluster Docker Swarm

Pour les clusters Swarm, un réseau dédié permet la communication entre nœuds :

```

1 void createSwarmNetwork(const std::string& username,
2                        const std::string& clusterName) {
3     std::string networkName = "swarm_" + username + "_" + clusterName;
4     std::string subnet = allocateSwarmSubnet(username, clusterName);
5
6     // Configuration similaire mais subnet diff rent
7     // Ex: 10.100.X.0/24 pour les clusters Swarm
8 }

```

Listing 6 – Réseau pour cluster Swarm

3.4 Système de Flavors

Flavors Implémentés Le système propose trois offres de ressources inspirées des cloud providers, avec pricing fictif en FCFA.

3.4.1 Définition des Flavors

```

1 struct Flavor {
2     std::string id;
3     std::string name;
4     int vcpu;
5     int ram_mb;

```

```

6     int disk_gb;
7     int price_monthly;
8 };
9
10 std::vector<Flavor> getFlavors() {
11     return {
12         {"small", "S (Small)", 1, 2048, 15, 2500},
13         {"medium", "M (Medium)", 2, 4096, 20, 3500},
14         {"large", "L (Large)", 4, 8192, 40, 6500}
15     };
16 }

```

Listing 7 – Structure Flavor (flavor_manager.hpp)

Flavor	vCPU	RAM	Disque	Prix/mois
Small (S)	1	2 Go	15 Go	2 500 FCFA
Medium (M)	2	4 Go	20 Go	3 500 FCFA
Large (L)	4	8 Go	40 Go	6 500 FCFA

TABLE 2 – Offres de ressources (Flavors)

3.4.2 Vérification de Capacité

Avant de créer une VM, le système vérifie la capacité disponible :

```

1 bool hasCapacity(virConnectPtr conn, const Flavor& flavor) {
2     unsigned long freeMemory = virNodeGetFreeMemory(conn);
3     virNodeInfo nodeInfo;
4     virNodeGetInfo(conn, &nodeInfo);
5
6     int freeCPU = nodeInfo.cpus - getUsedCPU(conn);
7
8     return (freeCPU >= flavor.vcpu &&
9             freeMemory >= (flavor.ram_mb * 1024 * 1024));
10 }

```

Listing 8 – Vérification de capacité

3.5 Gestion Multi-Hôtes

Objectif Le backend gère un pool de plusieurs hôtes KVM et sélectionne automatiquement l'hôte cible lors de la création d'une VM.

3.5.1 Configuration des Hôtes

```

1 {
2     "hosts": [
3         {
4             "id": "localhost",
5             "uri": "qemu:///system",
6             "hostname": "localhost",

```

```

7     "enabled": true
8 },
9 {
10    "id": "kvm-node-1",
11    "uri": "qemu+ssh://vps@192.168.1.100/system",
12    "hostname": "kvm-node-1",
13    "enabled": true
14 },
15 {
16    "id": "kvm-node-2",
17    "uri": "qemu+ssh://admin@10.0.0.50/system",
18    "hostname": "kvm-node-2",
19    "enabled": true
20 }
21 ]
22 }

```

Listing 9 – Configuration des hôtes (hosts.json)

3.5.2 Stratégie Least Used (LRU)

Stratégie implémentée : Sélectionner l'hôte avec le moins de charge (CPU + RAM utilisés).

```

1 std::string selectHostLRU(const std::vector<Host>& hosts) {
2     std::string bestHostId;
3     float lowestUsage = 100.0f;
4
5     for (const auto& host : hosts) {
6         if (!host.enabled) continue;
7
8         virConnectPtr conn = virConnectOpen(host.uri.c_str());
9         if (!conn) continue;
10
11         float cpuUsage = getCPUUsage(conn);
12         float memUsage = getMemoryUsage(conn);
13         float totalUsage = (cpuUsage + memUsage) / 2.0f;
14
15         if (totalUsage < lowestUsage) {
16             lowestUsage = totalUsage;
17             bestHostId = host.id;
18         }
19
20         virConnectClose(conn);
21     }
22
23     return bestHostId;
24 }

```

Listing 10 – Sélection d'hôte (Least Used)

3.5.3 Autres Stratégies Proposées

1. Round Robin :

- Principe : Sélectionner les hôtes à tour de rôle de manière circulaire
- Avantage : Distribution équitable, simple à implémenter

- Inconvénient : Ne tient pas compte de la charge réelle des hôtes
- Cas d'usage : Environnements homogènes avec charge prévisible

2. Random :

- Principe : Sélectionner un hôte aléatoirement parmi ceux disponibles
- Avantage : Très simple, pas de gestion d'état
- Inconvénient : Distribution non uniforme possible, pas optimal
- Cas d'usage : Tests, prototypes, charge très faible

3. Resource-Based :

- Principe : Sélectionner l'hôte avec le plus de ressources correspondant au flavor demandé
- Avantage : Optimise l'utilisation des ressources spécifiques
- Inconvénient : Plus complexe à calculer
- Cas d'usage : Hôtes hétérogènes, charges variables

4 Partie 2 : Déploiement Automatique de Clusters Docker Swarm

4.1 Architecture du Cluster Swarm

Fonctionnalité Les utilisateurs peuvent déployer leur propre cluster Docker Swarm en spécifiant le nombre de managers et workers. Le système déploie automatiquement les VMs et configure le cluster.

4.1.1 Configuration Type

Pour une démonstration, nous déployons un cluster avec :

- **1 Manager** : Nœud maître qui gère l'orchestration
- **2 Workers** : Nœuds qui exécutent les conteneurs
- **Réseau dédié** : 10.100.X.0/24 pour communication inter-nœuds

4.2 Déploiement Automatisé des VMs

4.2.1 Script de Déploiement

```
1 void deploySwarmCluster(const std::string& username,
2                          int nbManagers, int nbWorkers) {
3     std::string clusterName = "swarm_" + username;
4     std::string networkName = createSwarmNetwork(username, clusterName);
5
6     // Déployer les managers
7     for (int i = 0; i < nbManagers; i++) {
8         std::string vmName = clusterName + "_manager_" + std::to_string(
9             i);
10        deploySwarmNode(username, vmName, "manager", networkName);
11    }
```

```

12 // D ployer les workers
13 for (int i = 0; i < nbWorkers; i++) {
14     std::string vmName = clusterName + "_worker_" + std::to_string(i
15 );
16     deploySwarmNode(username, vmName, "worker", networkName);
17 }

```

Listing 11 – Création du cluster Swarm

4.3 Configuration Automatique via Cloud-Init

4.3.1 User-Data pour Installation Docker

```

1 #cloud-init
2 hostname: swarm-manager-0
3 users:
4   - name: swarm
5     sudo: ALL=(ALL) NOPASSWD:ALL
6     shell: /bin/bash
7     ssh_authorized_keys:
8       - ssh-rsa AAAAB3...
9
10 packages:
11   - docker.io
12   - docker-compose
13
14 runcmd:
15   # D marrer Docker
16   - systemctl enable docker
17   - systemctl start docker
18
19   # Ajouter l'utilisateur au groupe docker
20   - usermod -aG docker swarm
21
22   # Ouvrir les ports n cessaires pour Swarm
23   - ufw allow 2377/tcp # Cluster management
24   - ufw allow 7946/tcp # Node communication
25   - ufw allow 7946/udp # Node communication
26   - ufw allow 4789/udp # Overlay network
27
28   # Pour le manager: initialiser Swarm
29   - docker swarm init --advertise-addr $(hostname -I | awk '{print $1}')
30
31   # Sauvegarder le token worker
32   - docker swarm join-token worker -q > /tmp/worker-token

```

Listing 12 – Cloud-init user-data

4.3.2 Configuration des Workers

Pour les workers, une fois le manager initialisé :

```

1 #!/bin/bash
2 # R cup rer le token et l'IP du manager
3 MANAGER_IP="10.100.1.10"

```

```

4 WORKER_TOKEN=$(ssh swarm@$MANAGER_IP cat /tmp/worker-token)
5
6 # Rejoindre le cluster
7 docker swarm join --token $WORKER_TOKEN $MANAGER_IP:2377

```

Listing 13 – Rejoindre le cluster (worker)

4.4 Ports Nécessaires

Port	Protocole	Usage
2377	TCP	Cluster management
7946	TCP/UDP	Communication entre nœuds
4789	UDP	Overlay network (ingress)

TABLE 3 – Ports Docker Swarm

4.5 Affichage des Informations du Cluster

Une fois déployé, le système affiche :

```

1 Cluster Swarm d ploy avec succ s !
2
3 N uds :
4   - swarm-manager-0 (Manager) : 10.100.1.10
5   - swarm-worker-0 (Worker)   : 10.100.1.11
6   - swarm-worker-1 (Worker)   : 10.100.1.12
7
8 Commandes utiles:
9   # Lister les n uds
10  ssh swarm@10.100.1.10 docker node ls
11
12   # Initialiser le swarm (d j fait)
13  docker swarm init --advertise-addr 10.100.1.10
14
15   # Token pour ajouter des workers
16  docker swarm join-token worker

```

Listing 14 – Exemple de sortie

5 Partie 3 : Service PaaS - Catalogue d'Applications

5.1 Catalogue d'Applications

Applications Disponibles Le catalogue propose 8 applications prêtes à être déployées en 1 clic sur le cluster Docker Swarm de l'utilisateur.

5.1.1 Liste des Applications

1. **WordPress** : CMS populaire (PHP + MariaDB)

2. **Odoo** : ERP/CRM (Python + PostgreSQL)
3. **Mattermost** : Plateforme de messagerie d'équipe (PostgreSQL)
4. **OwnCloud** : Stockage et partage de fichiers (MariaDB)
5. **OnlyOffice** : Suite bureautique collaborative (PostgreSQL)
6. **OpenLDAP** : Serveur d'annuaire
7. **Moodle** : Plateforme d'apprentissage en ligne (MariaDB/PostgreSQL)
8. **PrestaShop** : E-commerce (MariaDB)

5.2 Mutualisation des Bases de Données

Optimisation Pour réduire la consommation de ressources, les instances PostgreSQL et MariaDB sont partagées entre tous les utilisateurs, avec isolation au niveau base de données.

5.2.1 Déploiement des SGBD Mutualisés

```
1 version: '3.8'
2 services:
3   postgres-shared:
4     image: postgres:15
5     environment:
6       POSTGRES_PASSWORD: ${POSTGRES_ROOT_PASSWORD}
7     volumes:
8       - postgres-data:/var/lib/postgresql/data
9     networks:
10      - paas-network
11     deploy:
12       replicas: 1
13       placement:
14         constraints: [node.role == manager]
15
16 volumes:
17   postgres-data:
18
19 networks:
20   paas-network:
21     driver: overlay
```

Listing 15 – Docker Compose pour PostgreSQL mutualisé

5.2.2 Création de Base par Application

Lors du déploiement d'une application :

```
1 void provisionDatabase(const std::string& appName,
2                       const std::string& username,
3                       const std::string& dbType) {
4   std::string dbName = username + "_" + appName;
5   std::string dbUser = username + "_" + appName + "_user";
6   std::string dbPassword = generateSecurePassword();
7 }
```



```

8     if (dbType == "postgresql") {
9         std::string cmd =
10             "docker exec postgres-shared psql -U postgres -c \" " +
11             "CREATE DATABASE " + dbName + "; " +
12             "CREATE USER " + dbUser + " WITH PASSWORD '" + dbPassword +
13             "'; " +
14             "GRANT ALL PRIVILEGES ON DATABASE " + dbName + " TO " +
15             dbUser + ";\n";
16         executeOnSwarm(cmd);
17     } else if (dbType == "mariadb") {
18         // Similaire pour MariaDB
19     }
20
21     // Sauvegarder les credentials de manière sécurisée
22     storeDBCredentials(username, appName, dbName, dbUser, dbPassword);
23 }

```

Listing 16 – Création de DB pour application

5.3 Déploiement en 1 Clic

5.3.1 Processus de Déploiement

1. Utilisateur sélectionne une application du catalogue
2. Système provisionne la base de données si nécessaire
3. Docker Compose personnalisé est généré
4. Stack est déployée sur le cluster Swarm
5. URL d'accès est fournie à l'utilisateur

5.3.2 Exemple : Déploiement WordPress

```

1 void deployWordPress(const std::string& username) {
2     // 1. Provisionner la base de données
3     auto dbInfo = provisionDatabase(username, "wordpress", "mariadb");
4
5     // 2. Générer docker-compose.yml
6     std::string composeContent = R"(
7 version: '3.8'
8 services:
9     wordpress:
10         image: wordpress:latest
11         environment:
12             WORDPRESS_DB_HOST: mariadb-shared
13             WORDPRESS_DB_NAME: )" + dbInfo.name + R"(
14             WORDPRESS_DB_USER: )" + dbInfo.user + R"(
15             WORDPRESS_DB_PASSWORD: )" + dbInfo.password + R"(
16         ports:
17             - )" + assignPort(username, "wordpress") + R"(:80"
18         networks:
19             - paas-network
20         deploy:
21             replicas: 1
22
23 networks:

```

```

24     paas-network:
25         external: true
26 );
27
28 // 3. Deployer
29 std::string stackName = username + "_wordpress";
30 deployStack(stackName, composeContent);
31
32 // 4. Retourner l'URL
33 return "http://" + getSwarmIP() + ":" + assignedPort;
34 }

```

Listing 17 – Déploiement WordPress

5.4 Système de Facturation

Méthode de Facturation La facturation est basée sur la consommation réelle des ressources : CPU, RAM, Disque et Temps d'utilisation.

5.4.1 Collecte des Métriques

```

1 struct ResourceUsage {
2     std::string username;
3     std::string appName;
4     float cpu_hours;
5     float ram_gb_hours;
6     float disk_gb;
7     int hours_running;
8 };
9
10 ResourceUsage collectMetrics(const std::string& stackName) {
11     // Récupérer les stats Docker
12     auto stats = getDockerStats(stackName);
13
14     ResourceUsage usage;
15     usage.cpu_hours = stats.cpu_time_seconds / 3600.0;
16     usage.ram_gb_hours = (stats.memory_bytes / (1024*1024*1024))
17                         * (stats.uptime_seconds / 3600.0);
18     usage.disk_gb = stats.disk_bytes / (1024*1024*1024);
19     usage.hours_running = stats.uptime_seconds / 3600;
20
21     return usage;
22 }

```

Listing 18 – Collecte des métriques de consommation

5.4.2 Calcul du Coût

```

1 struct Pricing {
2     float cpu_hour_price = 50.0;           // FCFA par heure de vCPU
3     float ram_gb_hour_price = 20.0;        // FCFA par Go-heure
4     float disk_gb_price = 100.0;           // FCFA par Go de stockage
5 };

```

```

6
7 float calculateCost(const ResourceUsage& usage, const Pricing& pricing)
8 {
9     float cpuCost = usage.cpu_hours * pricing.cpu_hour_price;
10    float ramCost = usage.ram_gb_hours * pricing.ram_gb_hour_price;
11    float diskCost = usage.disk_gb * pricing.disk_gb_price;
12
13    return cpuCost + ramCost + diskCost;
14 }

```

Listing 19 – Calcul de facturation

Ressource	Prix Unitaire
1 heure de vCPU	50 FCFA
1 Go-heure de RAM	20 FCFA
1 Go de stockage	100 FCFA

TABLE 4 – Grille tarifaire PaaS

6 Partie 4 : Service SaaS - Plateforme Collaborative

6.1 Architecture OwnCloud + OnlyOffice

Alternative Google Workspace La plateforme SaaS combine OwnCloud (stockage, partage) et OnlyOffice (édition collaborative) pour offrir une expérience similaire à Google Drive + Docs.

6.1.1 Fonctionnalités Proposées

- **Stockage de fichiers** : Upload, organisation en dossiers, corbeille
- **Partage** : Partage de fichiers/dossiers avec liens publics ou utilisateurs
- **Édition collaborative** : Documents texte, tableurs, présentations en temps réel
- **Gestion utilisateurs** : Comptes, groupes, permissions
- **Synchronisation** : Client desktop et mobile pour sync automatique
- **Versionning** : Historique des versions de fichiers

6.2 Déploiement sur Docker Swarm

6.2.1 Docker Compose

```

1 version: '3.8'
2
3 services:
4   owncloud:
5     image: owncloud/server:latest
6     environment:
7       OWNCLOUD_DOMAIN: cloud.thoth.local

```

```

8     OWNCLOUD_DB_TYPE: mysql
9     OWNCLOUD_DB_HOST: mariadb-shared
10    OWNCLOUD_DB_NAME: owncloud_db
11    OWNCLOUD_DB_USERNAME: owncloud_user
12    OWNCLOUD_DB_PASSWORD: ${DB_PASSWORD}
13    OWNCLOUD_ADMIN_USERNAME: admin
14    OWNCLOUD_ADMIN_PASSWORD: ${ADMIN_PASSWORD}
15    volumes:
16      - owncloud-data:/mnt/data
17    ports:
18      - "8080:8080"
19    networks:
20      - paas-network
21    deploy:
22      replicas: 1
23
24    onlyoffice:
25      image: onlyoffice/documentserver:latest
26      environment:
27        JWT_ENABLED: "true"
28        JWT_SECRET: ${JWT_SECRET}
29      ports:
30        - "8081:80"
31      networks:
32        - paas-network
33      deploy:
34        replicas: 1
35
36    volumes:
37      owncloud-data:
38
39    networks:
40      paas-network:
41        external: true

```

Listing 20 – Stack OwnCloud + OnlyOffice

6.3 Intégration OwnCloud - OnlyOffice

6.3.1 Configuration de l'Application OnlyOffice dans OwnCloud

Une fois les deux services déployés, configurer l'intégration :

```

1 # 1. Installer l'app OnlyOffice dans OwnCloud
2 occ app:install onlyoffice
3
4 # 2. Configurer l'adresse du serveur OnlyOffice
5 occ config:system:set onlyoffice DocumentServerUrl \
6   --value="http://onlyoffice:80/"
7
8 # 3. Configurer le JWT secret
9 occ config:system:set onlyoffice jwt_secret \
10  --value="${JWT_SECRET}"

```

Listing 21 – Configuration de l'intégration

6.4 Facturation au Stockage

Modèle de Facturation SaaS La facturation est basée sur l'espace de stockage utilisé, avec une tarification progressive par paliers.

6.4.1 Grille Tarifaire

Stockage	Prix mensuel
0 - 10 Go	Gratuit
10 - 50 Go	1 000 FCFA
50 - 100 Go	2 000 FCFA
100 - 500 Go	5 000 FCFA
500 Go - 1 To	10 000 FCFA
> 1 To	15 000 FCFA + 10 FCFA/Go supplémentaire

TABLE 5 – Tarification SaaS au stockage

6.4.2 Calcul de la Consommation

```

1 float calculateStorageCost(float storageGB) {
2     if (storageGB <= 10) return 0;
3     if (storageGB <= 50) return 1000;
4     if (storageGB <= 100) return 2000;
5     if (storageGB <= 500) return 5000;
6     if (storageGB <= 1024) return 10000;
7
8     // Plus de 1 To
9     float excess = storageGB - 1024;
10    return 15000 + (excess * 10);
11 }
```

Listing 22 – Calcul du coût de stockage

7 Implémentation Frontend

7.1 Structure de l'Interface

L'interface web est organisée en vues multiples accessibles via une navigation latérale :

- **Dashboard** : Vue d'ensemble des ressources
- **VMs** : Gestion des machines virtuelles
- **Deploy VM** : Formulaire de création de VM avec flavors
- **Docker Swarm** : Gestion des clusters
- **PaaS Apps** : Catalogue et déploiement d'applications
- **Networks** : Gestion des réseaux
- **Hosts** : Administration multi-hôtes (admin)
- **Billing** : Facturation et consommation
- **Users** : Administration utilisateurs (admin)

7.2 Authentification et Session

7.2.1 Système JWT

```
1 async function login(username, password) {
2   const response = await fetch('http://localhost:3000/api/auth/login',
3     {
4       method: 'POST',
5       headers: {'Content-Type': 'application/json'},
6       body: JSON.stringify({username, password})
7     });
8   if (response.ok) {
9     const data = await response.json();
10    localStorage.setItem('token', data.token);
11    localStorage.setItem('username', data.username);
12    localStorage.setItem('role', data.role);
13    window.location.href = '/index.html';
14  } else {
15    showError('Identifiants invalides');
16  }
17 }
```

Listing 23 – Authentification JWT (auth.js)

7.2.2 Protection des Routes

```
1 function checkAuth() {
2   const token = localStorage.getItem('token');
3   if (!token) {
4     window.location.href = '/login.html';
5     return false;
6   }
7
8   // Vérifier la validité du token
9   fetch('http://localhost:3000/api/auth/verify', {
10     headers: {'Authorization': 'Bearer ${token}' }
11   }).then(response => {
12     if (!response.ok) {
13       logout();
14     }
15   });
16
17   return true;
18 }
```

Listing 24 – Vérification d'authentification

7.3 Gestion des VMs

7.3.1 Affichage des VMs

```
1 async function loadVMs() {
2   const token = localStorage.getItem('token');
3   const response = await fetch('http://localhost:3000/api/vms', {
```

```
4     headers: {'Authorization': 'Bearer ${token}'}
5   });
6
7   const data = await response.json();
8   const vmsList = document.getElementById('vms-list');
9   vmsList.innerHTML = '';
10
11   data.vms.forEach(vm => {
12     const card = createVMCard(vm);
13     vmsList.appendChild(card);
14   });
15 }
16
17 function createVMCard(vm) {
18   const card = document.createElement('div');
19   card.className = 'vm-card';
20   card.innerHTML = `
21     <h3>${vm.display_name}</h3>
22     <span class="status ${vm.state}">${vm.state}</span>
23     <p>Flavor: ${vm.flavor}</p>
24     <p>IP: ${vm.ip || 'N/A'}</p>
25     <div class="actions">
26       <button onclick="startVM('${vm.name}')">Start</button>
27       <button onclick="stopVM('${vm.name}')">Stop</button>
28       <button onclick="deleteVM('${vm.name}')">Delete</button>
29     </div>
30   `;
31   return card;
32 }
```

Listing 25 – Chargement des VMs (vms.js)

7.4 Déploiement de VMs avec Flavors

```
1 async function deployVM(event) {
2   event.preventDefault();
3
4   const vmData = {
5     vmName: document.getElementById('vm-name').value,
6     flavor: document.getElementById('flavor').value,
7     osImage: document.getElementById('os-image').value,
8     cloudInitUser: document.getElementById('cloud-init-user').value,
9     cloudInitPassword: document.getElementById('cloud-init-password
10   ').value
11   };
12
13   const token = localStorage.getItem('token');
14   const response = await fetch('http://localhost:3000/api/vms/deploy',
15   {
16     method: 'POST',
17     headers: {
18       'Content-Type': 'application/json',
19       'Authorization': 'Bearer ${token}'
20     },
21     body: JSON.stringify(vmData)
22   });
```

```
22     if (response.ok) {
23         showSuccess('VM en cours de d ploiement...');
24         setTimeout(loadVMs, 3000);
25     } else {
26         const error = await response.json();
27         showError(error.message);
28     }
29 }
```

Listing 26 – Formulaire de déploiement

7.5 Catalogue PaaS

7.5.1 Affichage du Catalogue

```
1  const catalog = [
2      {
3          id: 'wordpress',
4          name: 'WordPress',
5          description: 'CMS le plus populaire au monde',
6          icon: 'wordpress-icon.png',
7          dbType: 'mariadb'
8      },
9      {
10         id: 'odoo',
11         name: 'Odoo',
12         description: 'Suite ERP/CRM compl te',
13         icon: 'odoo-icon.png',
14         dbType: 'postgresql'
15     },
16     // ... autres applications
17 ];
18
19 function renderCatalog() {
20     const grid = document.getElementById('catalog-grid');
21     grid.innerHTML = catalog.map(app => `
22         <div class="app-card" onclick="deployApp('${app.id}')">
23             
24             <h3>${app.name}</h3>
25             <p>${app.description}</p>
26             <button>D ployer </button>
27         </div>
28     `).join('');
29 }
```

Listing 27 – Catalogue d'applications

7.6 Thème Clair/Sombre

```
1  function toggleTheme() {
2      const currentTheme = localStorage.getItem('theme') || 'light';
3      const newTheme = currentTheme === 'light' ? 'dark' : 'light';
4
5      document.body.classList.remove('theme-${currentTheme}');
6      document.body.classList.add('theme-${newTheme}');
```



```
7
8     localStorage.setItem('theme', newTheme);
9 }
10
11 // Appliquer le th me au chargement
12 document.addEventListener('DOMContentLoaded', () => {
13     const theme = localStorage.getItem('theme') || 'light';
14     document.body.classList.add('theme-${theme}');
15 });
```

Listing 28 – Gestion des thèmes (theme.js)

8 Guide de Déploiement

8.1 Prérequis

- Ubuntu 22.04 LTS ou supérieur
- Support de virtualisation matérielle (Intel VT-x ou AMD-V)
- Au moins 8 Go de RAM et 50 Go d'espace disque
- Docker et Docker Compose installés
- Accès root ou sudo

8.2 Installation

8.2.1 Étape 1 : Cloner le Dépôt

```
1 git clone https://github.com/InputOutputStream/tp_kvm.git
2 cd thoth-cloud
```

8.2.2 Étape 2 : Configuration

```
1 # Copier le fichier de configuration exemple
2 cp .env.example .env
3
4 # diter les variables d'environnement
5 nano .env
6
7 # Configurer les h tes KVM
8 cp config/hosts.json.example config/hosts.json
9 nano config/hosts.json
```

8.2.3 Étape 3 : Compilation du Backend

```
1 # Installer les d pendances
2 sudo apt install build-essential cmake libvirt-dev \
3     libssh2-1-dev nlohmann-json3-dev
4
5 # Compiler
6 mkdir build && cd build
7 cmake ..
```

```
8 make
9 cd ..
```

8.2.4 Étape 4 : Déploiement avec Docker

```
1 # Build de l'image
2 docker build -t thoth-cloud:latest .
3
4 # Lancement avec docker-compose
5 docker-compose up -d
```

8.3 Accès à l'Interface

Une fois déployé, accéder à l'interface web :

```
1 http://localhost:8080
2
3 Credentials par défaut :
4   Username: admin
5   Password: admin123 (changer immédiatement)
```

8.4 Configuration Post-Installation

1. Changer le mot de passe admin
2. Configurer les hôtes KVM distants
3. Créer les réseaux utilisateurs
4. Initialiser le cluster Swarm pour PaaS
5. Déployer les bases de données mutualisées

9 Résultats et Démonstrations

9.1 Partie 1 : IaaS Multi-Tenant

Résultats Obtenus

- Isolation complète des ressources par utilisateur
- Système de nommage unique empêchant les conflits
- Backend conteneurisé communiquant avec hôtes distants
- Réseaux isolés par utilisateur et par cluster
- 3 flavors avec vérification de capacité
- Gestion multi-hôtes avec stratégie LRU

9.2 Partie 2 : Clusters Docker Swarm

Résultats Obtenus

- Déploiement automatique de clusters (1 manager + 2 workers)
- Configuration automatique via cloud-init
- Docker installé et configuré automatiquement
- Ports Swarm ouverts correctement
- Affichage des IPs et commandes de gestion

9.3 Partie 3 : PaaS et Catalogue

Résultats Obtenus

- Catalogue de 8 applications fonctionnel
- Déploiement en 1 clic opérationnel
- Bases de données PostgreSQL et MariaDB mutualisées
- Isolation logique par base de données
- Système de facturation implémenté
- Affichage de l'URL d'accès après déploiement

9.4 Partie 4 : SaaS Collaboratif

Résultats Obtenus

- OwnCloud + OnlyOffice déployés et intégrés
- Stockage, partage et édition collaborative fonctionnels
- Gestion multi-utilisateurs
- Facturation au stockage implémentée
- Interface similaire à Google Workspace

10 Limitations et Améliorations

10.1 Limitations Connues

Points d'Attention

- **Sécurité** : Authentification basique, pas de 2FA ni OAuth
- **Monitoring** : Métriques limitées, pas de système d'alerting
- **Backups** : Pas de système de backup automatique des VMs/données
- **Haute Disponibilité** : Backend single-instance, pas de failover
- **Scaling** : Pas d'auto-scaling des applications PaaS

- **Réseau** : Configuration réseau simplifiée, pas de VLAN avancé
- **Logs** : Logging basique, pas d'agrégation centralisée

10.2 Améliorations Proposées

10.2.1 Court Terme (1-3 mois)

1. Sécurité :

- Implémenter OAuth2/OpenID Connect
- Ajouter 2FA (TOTP)
- Chiffrement des données sensibles
- Rate limiting et protection DDoS

2. Monitoring :

- Intégration Prometheus + Grafana
- Dashboards de monitoring en temps réel
- Système d'alerting (email, Slack)
- Logs centralisés (ELK stack)

3. Backups :

- Snapshots automatiques des VMs
- Backup incrémental des données
- Politique de rétention configurable
- Restauration en 1 clic

10.2.2 Moyen Terme (3-6 mois)

1. Haute Disponibilité :

- Backend multi-instances avec load balancing
- Shared storage (Ceph ou GlusterFS)
- Failover automatique
- Réplication des données

2. Kubernetes :

- Support K8s en plus de Docker Swarm
- Helm charts pour applications
- Operators personnalisés
- Multi-cluster management

3. Networking Avancé :

- SDN (Software-Defined Networking)
- Support VLAN
- Firewall programmable
- VPN inter-sites

10.2.3 Long Terme (6-12 mois)

1. **AI/ML** :
 - Auto-scaling prédictif basé sur ML
 - Détection d'anomalies
 - Optimisation automatique des ressources
 - Prédiction des coûts
2. **Marketplace** :
 - Catalogue d'images communautaires
 - Templates d'infrastructure as code
 - Validation et certification automatique
 - Système de rating et reviews
3. **Compliance** :
 - Audit trails complets
 - GDPR compliance tools
 - Encryption at rest et in transit
 - Security scanning automatique

11 Conclusion

11.1 Bilan de Réalisation

Ce projet THOTH CLOUD a permis de développer une plateforme cloud complète et fonctionnelle intégrant les trois couches de services cloud : IaaS, PaaS et SaaS. L'ensemble des objectifs du TP4 ont été atteints avec succès.

Objectifs Atteints **Partie 1 - IaaS** : Multi-tenant, Découplage, Isolation réseau, Flavors, Multi-hôtes
Partie 2 - Swarm : Déploiement automatique, Configuration cloud-init
Partie 3 - PaaS : Catalogue 8 apps, DB mutualisées, Facturation
Partie 4 - SaaS : OwnCloud + OnlyOffice, Alternative Google Workspace

11.2 Compétences Développées

Ce travail pratique a permis de développer des compétences approfondies en :

- **Architecture cloud** : Conception de systèmes multi-services IaaS/PaaS/SaaS
- **Programmation système** : Développement backend en C++ avec libvirt
- **Virtualisation** : Maîtrise avancée de KVM/QEMU et gestion multi-hôtes
- **Conteneurisation** : Docker et orchestration avec Docker Swarm
- **Développement web** : Interface moderne SPA avec JavaScript
- **Réseaux virtuels** : Isolation et gestion de réseaux avec libvirt
- **DevOps** : Automatisation avec cloud-init, Ansible, Docker Compose
- **Sécurité** : Authentification JWT, isolation multi-tenant
- **Facturation** : Modélisation de systèmes de billing

11.3 Perspectives d'Évolution

THOTH CLOUD constitue une base solide pour un cloud de production. La roadmap d'améliorations permettra d'atteindre la maturité d'un cloud professionnel :

- Renforcement de la sécurité et de la conformité
- Haute disponibilité et résilience
- Monitoring et observabilité avancés
- Support Kubernetes pour orchestration moderne
- Intelligence artificielle pour optimisation
- Marketplace communautaire d'applications

11.4 Remerciements

Je tiens à remercier :

- Dr. Valery MONTHE et M. Gildas NGOUANFO pour leur encadrement et leurs conseils techniques
- La communauté open-source pour les outils utilisés (libvirt, Docker, etc.)
- Mes camarades de promotion pour les échanges enrichissants

THOTH CLOUD
Cloud Computing for Everyone
IaaS · PaaS · SaaS

A Structure du Code Source

```
1 thoth-cloud/  
2     backend/  
3         include/  
4             baseimage_manager.hpp  
5             config_manager.hpp  
6             flavor_manager.hpp  
7             host_manager.hpp  
8             network_manager.hpp  
9             paas_operations.hpp  
10            swarm_operations.hpp  
11            user_operations.hpp  
12            utils.hpp  
13            vm_operations.hpp  
14        src/  
15            main.cpp  
16            routes.cpp  
17            vm_operations.cpp  
18            host_manager.cpp  
19            network_manager.cpp
```

```
20         paas_operations.cpp
21         user_operations.cpp
22     CMakeLists.txt
23     Dockerfile
24 frontend/
25     index.html
26     login.html
27     register.html
28     styles/
29         main.css
30         auth.css
31         cards.css
32         sidebar.css
33         forms.css
34         modals.css
35         dark-theme.css
36     scripts/
37         auth.js
38         api-service.js
39         navigation.js
40         vms.js
41         deploy.js
42         paas-apps.js
43         swarm.js
44         networks.js
45         hosts.js
46         billing.js
47         theme.js
48     assets/
49         images/
50     scripts/
51         setup_remote.sh
52         paas-db-setup.sh
53         ownCloudSetup.sh
54     config/
55         hosts.json
56         flavors.json
57     docker-compose.yml
58     .env.example
59     nginx.conf
60     README.md
```

Listing 29 – Arborescence complète du projet

B Configuration Système

B.1 Fichier de Configuration Principal

```
1 # API Configuration
2 API_PORT=3000
3 API_HOST=0.0.0.0
4
5 # KVM Hosts
6 HOSTS=qemu:///system,qemu+ssh://vps@192.168.1.100/system
7
8 # Paths
```

```
9 BASE_IMAGE_DIR=/var/lib/libvirt/images/baseimg
10 CLOUD_INIT_DIR=/var/lib/libvirt/images/cloud-init-iso
11 VM_STORAGE_DIR=/var/lib/libvirt/images
12
13 # Database
14 POSTGRES_HOST=postgres-shared
15 POSTGRES_PORT=5432
16 MARIADB_HOST=mariadb-shared
17 MARIADB_PORT=3306
18
19 # Security
20 JWT_SECRET=your-secret-key-change-this
21 SESSION_TIMEOUT=3600
```

Listing 30 – .env

B.2 Configuration des Hôtes

```
1 {
2   "hosts": [
3     {
4       "id": "localhost",
5       "uri": "qemu:///system",
6       "hostname": "localhost",
7       "enabled": true,
8       "max_vms": 20,
9       "max_vcpus": 16,
10      "max_ram_gb": 32
11    },
12    {
13      "id": "kvm-node-1",
14      "uri": "qemu+ssh://vps@192.168.1.100/system",
15      "hostname": "kvm-node-1",
16      "enabled": true,
17      "max_vms": 30,
18      "max_vcpus": 24,
19      "max_ram_gb": 64
20    }
21  ],
22  "selection_strategy": "least_used"
23 }
```

Listing 31 – config/hosts.json

C Commandes Utiles

C.1 Gestion des VMs

```
1 # Lister toutes les VMs
2 virsh list --all
3
4 # Démarrer une VM
5 virsh start vm_name
6
```



```
7 # Arrêter proprement
8 virsh shutdown vm_name
9
10 # Forcer l'arrêt
11 virsh destroy vm_name
12
13 # Supprimer une VM
14 virsh undefine vm_name --remove-all-storage
15
16 # Voir les infos d'une VM
17 virsh dominfo vm_name
18
19 # Console série
20 virsh console vm_name
21
22 # Voir les réseaux
23 virsh net-list --all
```

C.2 Gestion Docker Swarm

```
1 # Initialiser le swarm
2 docker swarm init --advertise-addr <IP>
3
4 # Obtenir le token worker
5 docker swarm join-token worker
6
7 # Lister les nœuds
8 docker node ls
9
10 # Déployer un stack
11 docker stack deploy -c compose.yml stack_name
12
13 # Lister les services
14 docker service ls
15
16 # Voir les logs d'un service
17 docker service logs service_name
18
19 # Scaler un service
20 docker service scale service_name=3
21
22 # Supprimer un stack
23 docker stack rm stack_name
```

D Captures d'Écran

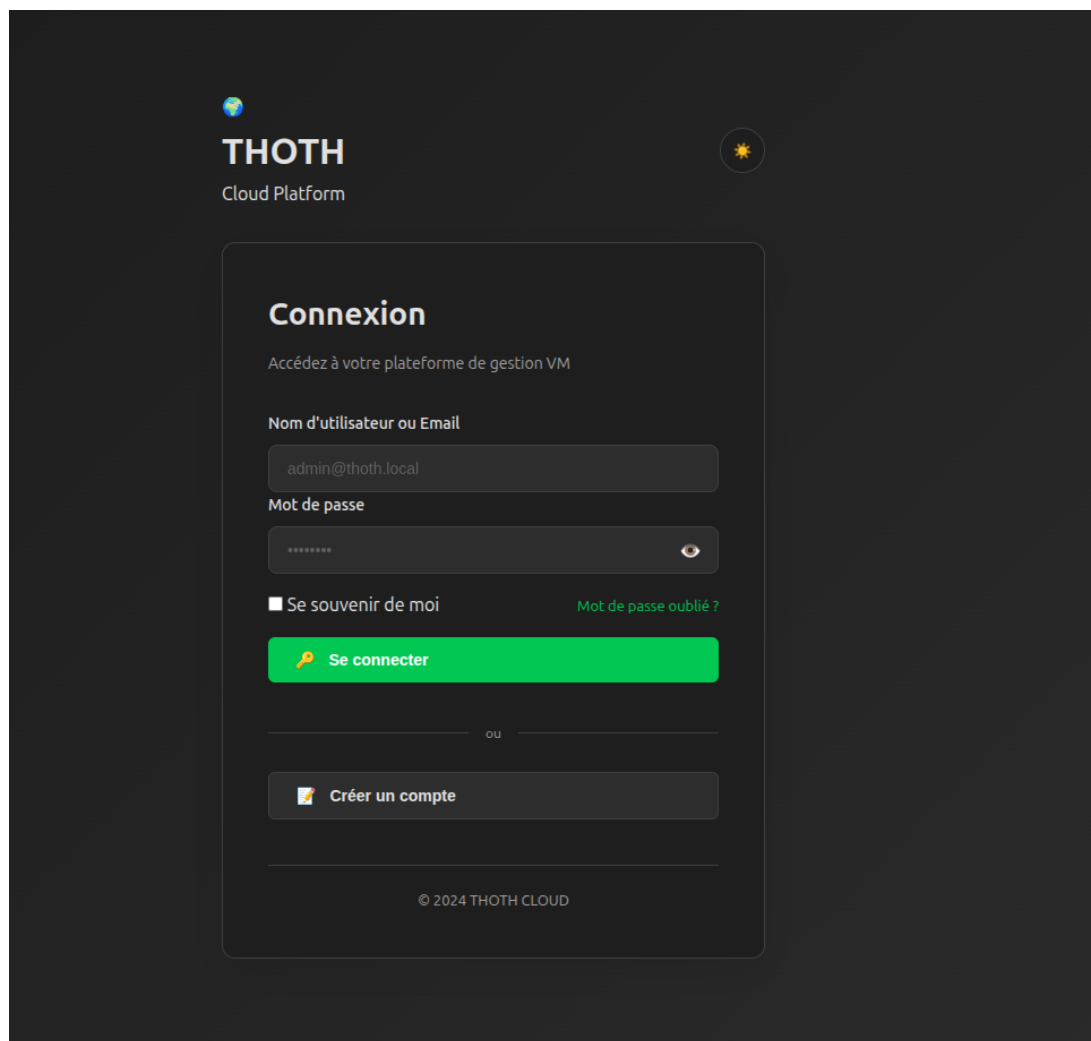


FIGURE 2 – Interface de connexion THOTH CLOUD

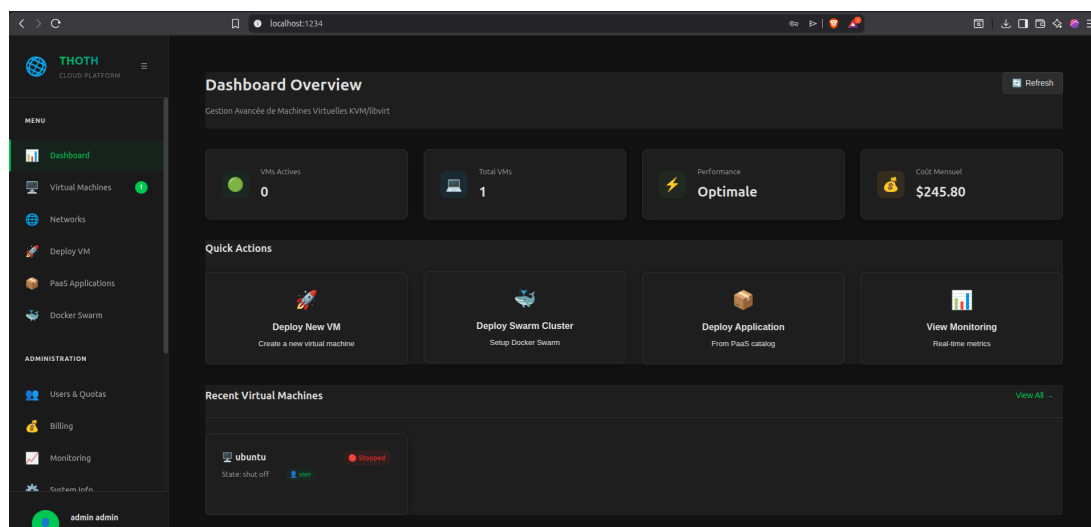


FIGURE 3 – Vue du tableau de bord

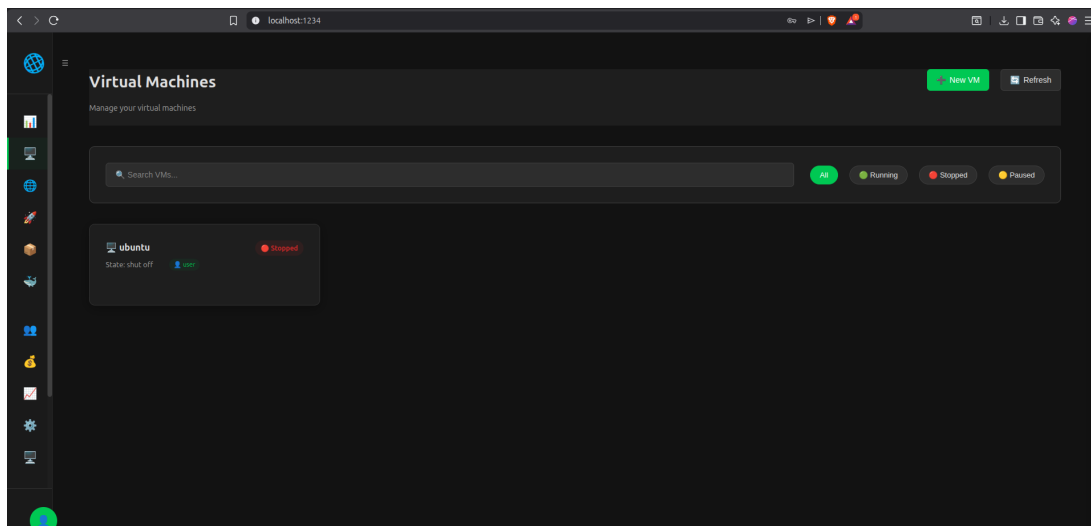


FIGURE 4 – Interface de gestion des machines virtuelles

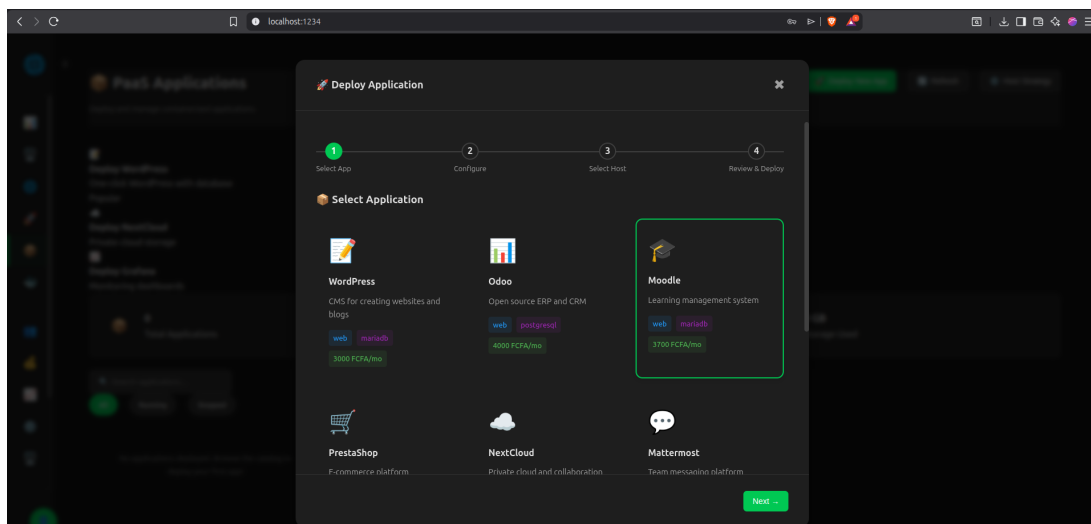


FIGURE 5 – Catalogue d'applications déployables